

1. How does the `useNavigate` hook differ from the `Link` component in `React Router`?

`useNavigate` Hook

- Used for **programmatic navigation** — you navigate through JavaScript logic (for example, after form submission or based on conditions).
- Works **inside React components** and **does not render any HTML**.

Example:

```
import { useNavigate } from "react-router-dom";

function Login() {

  const navigate = useNavigate();

  const handleLogin = () => {

    // perform login logic

    navigate("/dashboard"); // redirect programmatically

  };

  return <button onClick={handleLogin}>Login</button>;

}
```

`Link` Component

- Used for **declarative navigation** — renders a clickable link to another page.
- Works like an `<a>` tag but **without reloading the page**.

Example:

```
import { Link } from "react-router-dom";
```

```
function Navbar() {

  return (

    <nav>

      <Link to="/about">About</Link>

    </nav>

  );

}
```

2. Describe how you would implement dynamic form fields in React, where the number of fields can change based on user input.

To create form fields that **add or remove inputs dynamically**, use a **state array** to store field data.

Example:

```
import { useState } from "react";

function DynamicForm() {

  const [fields, setFields] = useState([ { value: "" } ]);

  // Add new field

  const addField = () => {

    setFields([...fields, { value: "" }]);

  };

  // Remove field
```

```
const removeField = (index) => {  
  const newFields = fields.filter((_, i) => i !== index);  
  setFields(newFields);  
};
```

```
// Handle input change
```

```
const handleChange = (index, event) => {  
  const newFields = [...fields];  
  newFields[index].value = event.target.value;  
  setFields(newFields);  
};
```

```
return (  
  <div>  
    <h2>Dynamic Form Fields</h2>  
    {fields.map((field, index) => (  
      <div key={index}>  
        <input  
          type="text"  
          value={field.value}  
          onChange={(e) => handleChange(index, e)}  
          placeholder={`Field ${index + 1}`}  
        />  
      )
```

```
        <button onClick={() => removeField(index)}>Remove</button>

      </div>

    )}

    <button onClick={addField}>Add Field</button>

  </div>

);

}
```

- fields holds all input states.
- addField() adds a new field to the array.
- removeField() deletes a specific input field.
- handleChange() updates the value of a field dynamically